

SCALECRAFT

The Crafters Guide to System Design

A short book about a long subject

QUICK REFERENCE

Twenty-two chapters, three checkpoints and five real systems — from three boxes and two arrows to a video service that must serve almost every byte from the edge. Each idea arrives at the moment the previous one has created a problem it can solve.

Parts

I-VIII + appendices

Components

All 27, each once

Read

In order, the first time

Contents

Eight parts · 23 chapters · 3 checkpoints · 5 real systems

How to read this guide	5
PART I — How the Web Serves a Request	
0.1 Client, Server, Database	7
0.2 Naming: DNS and the Browser	9
0.3 The Edge: Firewall and Reverse Proxy	10
PART II — Scaling Compute	
1.1 Vertical vs. Horizontal Scaling	12
1.2 Load Balancing	14
1.3 Statelessness and Sessions	15
1.4 Reverse Proxy vs. Load Balancer vs. API Gateway	16
PART III — Caching	
2.1 Cache-Aside	19
2.2 Distributed Caching	21
2.3 Caching at the Edge: CDN	22
Checkpoint R1 A Site That Stays Up	23
PART IV — Scaling Data	
3.1 Read Replicas and Replication	25
3.2 Choosing a Database: SQL vs. NoSQL	27
3.3 Blobs: Object Storage	29
3.4 Search: the Search Engine	30

3.5	Partitioning and Sharding	32
------------	----------------------------------	-----------

PART V — Asynchronous Work

4.1	Queues and Workers	35
4.2	When Work Fails: Dead Letter Queues	37
4.3	Scheduled and Ephemeral Compute	38
4.4	Streams and Events: Event Bus and Kafka	40
Checkpoint R2	The Whole Backend	41

PART VI — Distributed Coordination

5.1	Leaders and Followers	43
5.2	Agreement: Coordinators and Locks	45
5.3	Trade-offs at the Core: Consistency and Availability	47

PART VII — Designing Whole Systems

6.1	From Requirements to Architecture	50
Checkpoint R3	Open Brief	52

PART VIII — Five Real Systems

RWE-1	bit.ly, a URL Shortener	54
RWE-2	Instagram, Photo Sharing	56
RWE-3	A Distributed Log Collector	57
RWE-4	WhatsApp, Messaging	59
RWE-5	Netflix, Video Streaming	61

APPENDICES — Reference

Appendix A	The 27 Components	64
-------------------	--------------------------	-----------

Appendix B	Anti-Patterns	67
Appendix C	Reading the Trade-offs	70

How to read this guide

This is a short book about a long subject. It follows the ScaleCraft curriculum exactly — seven units, twenty-three chapters, three checkpoints and five real-world systems — but it is written to be

read

, not clicked through. Everything here is the reasoning you would want in your head before you drag your first box onto a canvas.

Three rules governed the writing.

Nothing appears before the problem that motivates it. Load balancers show up one chapter after you have felt the pain of two servers and no way to choose between them. Coordination comes last because it is about agreement, and agreement is only interesting once you have things that could disagree. If a chapter feels like it arrives at exactly the moment you were starting to ask its question, that is deliberate.

Every component is introduced once. There are twenty-seven of them and each has a home chapter. When one comes back it comes back doing a second job, which is usually the more interesting lesson — a distributed cache holding sessions, an object store fronted by a CDN. The appendix indexes all of them if you need to look one up in a hurry.

Being wrong is part of the method. Each chapter ends with a **Get it wrong** section describing the specific broken shape people build, and why it breaks. Read those even if you skim the rest — the failure modes are more memorable than the happy paths, and they are what a design review will actually catch you on.

The shape of the book

Parts I through VII are the building blocks, in strict order. Read them in sequence the first time. Part VIII is five complete systems — bit.ly, Instagram, a log collector, WhatsApp, Netflix — that recombine everything, and can be read in any order once you have finished Part VII. The three checkpoints between parts are not chapters; they are pauses that ask you to rebuild from memory what you have just been given.

What this book will not do

It will not teach you Raft, or the internals of any particular database, or a vendor's product line. It teaches you when you need coordination and what it costs you — not how consensus is implemented. That line is drawn deliberately and held throughout: this is a guide to judgment, and judgment is mostly knowing which question you are actually facing.

PART I

How the Web Serves a Request

Three boxes, two arrows, and one front door — the request path everything else decorates.

0.1 — Client, Server, Database

0.2 — Naming: DNS and the Browser

0.3 — The Edge: Firewall and Reverse Proxy

0.1 — Client, Server, Database

In one line: every system in this book is a decorated version of three boxes and two arrows.

A user opens something on their phone. Some code runs somewhere. Something is remembered. That is the whole of computing at this altitude, and the three components that carry it — **client**, **application server**, **database** — are the atoms you will still be arranging in the last chapter, when the diagram has forty boxes on it.

The **client** is the thing the user touches: a browser tab, a mobile app, another company's script calling your API. It is untrusted, always. It runs on hardware you do not control, and anything you ship to it — code, keys, assumptions — is now public.

The **application server** is where your product's rules live. It authenticates ("who is this?"), authorises ("are they allowed?"), validates ("is this input sane?"), and only then does it touch data. It exists precisely because the client cannot be trusted to do any of that honestly.

The **database** remembers. It enforces the invariants that must survive a crash: this username is unique, this balance is never negative, this order belongs to that customer.

Between them runs the **request/response cycle**, the pattern that dominates the first half of this book: a client asks, waits, and receives. The waiting is the important part. Everything in Part V exists because sometimes you should not make the user wait.

Why not let the client talk to the database?

This is the first real question, and worth answering carefully, because the reasoning generalises.

To connect to a database you need credentials. Ship them to the client and you have published them — obfuscation is not secrecy, and anyone with developer tools can read them in about fifteen seconds. Now every user has full read/write access to every row.

Even if credentials were free, the client cannot enforce your rules. "Only the owner may delete this post" is a rule your code checks. Delete your code from the path and only the attacker's code remains. Worse, your table layout becomes your public API: rename a column and you break every stale mobile app in the world.

The application server is the enforcement point. That is its entire justification, and it is why the shape is

always

client → server → database, never client → database.

Components introduced: `client`, `app-server`, `sql-database`. **Edge kind:** `request-flow`.

Get it wrong. A client wired straight to a database (`no-direct-client-database`). It works beautifully in a demo and is unfixable in production: credentials are exposed, no business rule is enforceable, and your schema is now a contract with strangers.

Remember

- Three tiers exist to put an enforcement point between an untrusted device and durable state.
- The client is untrusted by definition, not by suspicion.
- Request/response means the user is waiting — a cost you will spend the rest of this book learning to manage.

0.2 — Naming: DNS and the Browser

In one line: before a request can be served it has to find you, and finding you is a separate system.

Your architecture diagram starts at the client. Reality starts one step earlier: the client holds a name — `example.com` — and the network only routes to addresses. Something has to translate. That something is **DNS**, and it is the largest distributed database most engineers never think about.

When a **browser** is asked for `example.com` it walks a cache hierarchy before it does anything else: its own in-memory cache, then the operating system's, then the resolver run by the ISP or a public service, which in turn asks the root, the `.com` servers, and finally the nameservers you control. The answer comes back with a **TTL** — a number of seconds the answer may be reused. Only then does a connection get opened and the request from 0.1 actually happen.

Two consequences matter for design.

DNS is not on the data path. It is consulted before the request, not during it. A steady stream of traffic to your site does not generate a corresponding stream of DNS lookups, because the answers are cached at every level. This is your first encounter with a lesson Part IV formalises: not every arrow between two boxes is the same kind of arrow. Some carry requests, some carry answers to questions asked once and remembered.

That caching is also the trap. Because you do not control the caches, you cannot revoke an answer. Change a record with a one-hour TTL and some clients will keep the old address for an hour after you "fixed" it — some misbehaving resolvers, longer. This makes DNS a slow steering wheel: fine for planned moves, useless for reacting to an outage in seconds. Lower the TTL before you plan to change something, not after.

The upside is that a slow steering wheel is still a steering wheel, and it is the cheapest one you own. Returning different addresses to different askers — by geography, by health, by weight — is how a CDN puts a user on a nearby edge server, which is Chapter 2.3.

Components introduced: `browser`, `dns`.

Get it wrong. Treating DNS as a live routing layer — pointing a record at a single server and expecting a record change to fail traffic over quickly. It will not. Failover lives behind a load balancer (1.2); DNS steers at the coarse level, slowly.

Remember

- Resolution happens before connection, and is cached at every layer including the browser.
- TTL is a promise you cannot take back early; lower it in advance of changes.
- DNS steering is coarse and slow, which is exactly enough for CDNs and regions.

0.3 — The Edge: Firewall and Reverse Proxy

In one line: give the system exactly one front door, then put a guard on it.

You now have a name that resolves and a tier that serves. The question this chapter answers is

what should be reachable from the internet at all?

The correct answer is: almost nothing.

A **firewall** is the rule that decides. In practice it is a small allow-list: ports 80 and 443, from anywhere, to one address. Everything else — your database port, your admin interface, your internal metrics endpoint, SSH — is reachable only from inside. The instinct to write "allow all, we're behind a VPN anyway" is the single most common way a system that was fine on Friday is in a breach report on Monday, because it converts one compromised component into total access.

A **reverse proxy** is the front door itself. Clients connect to it and to nothing else; it forwards to whatever is actually running behind. "Reverse" is relative to a forward proxy: a forward proxy sits with the client and hides who is asking; a reverse proxy sits with the server and hides who is answering.

That indirection buys four things immediately. It **terminates TLS** in one place, so certificates live at one address instead of on every machine. It gives you **one public address** regardless of how many machines are behind it. It can **serve static files** without waking your application at all. And it lets you **change the inside of the system** — add machines, move a route to a new service, take a host out for patching — without a single client noticing, because the address they hold never changes.

That last property is the reason this chapter comes before any scaling chapter. Horizontal scaling is only invisible to clients if there was already something standing in front. Put the front door in first and Part II becomes a change to the inside of the house.

Components introduced: `firewall`, `reverse-proxy`.

Get it wrong. A permissive firewall (`permissive-firewall`) — a policy that allows everything from everywhere, usually written during debugging and never taken back. Note the shape of this mistake: the topology is perfect and the configuration is fatal. Architecture includes configuration, and a diagram cannot show you this one.

Remember

- One entry point, one TLS termination point, one address clients ever learn.
- Everything not deliberately public should be unreachable from outside.
- A correct diagram with a wrong config is a wrong system.

PART II

Scaling Compute

The gentlest scaling story: stateless machines are interchangeable, and interchangeable means you can simply add more.

1.1 — Vertical vs. Horizontal Scaling

1.2 — Load Balancing

1.3 — Statelessness and Sessions

1.4 — Reverse Proxy vs. Load Balancer vs. API Gateway

1.1 — Vertical vs. Horizontal Scaling

In one line: you can buy a bigger machine, or you can buy more machines, and the second choice constrains everything you build afterwards.

Traffic grows. The single application server from Unit 0 saturates — CPU pinned, requests queueing, latency climbing. There are exactly two moves.

Vertical scaling is a bigger box. More cores, more RAM, faster disk. Its enormous virtue is that nothing in your code changes: same process, same memory, same assumptions. Its virtue is also its limit. Machines have a largest size, and the price curve toward that size is brutally non-linear — the top-end instance often costs several times the mid-range one for perhaps twice the capacity. And the ceiling is not the real problem. The real problem is that one machine is one failure: it reboots, and you are entirely offline.

Horizontal scaling is more boxes. Capacity becomes additive and roughly linear in cost, which is pleasant, and no single machine's death is fatal, which matters more. In exchange you inherit a permanent tax: there is now more than one of everything, and any state living inside one process is invisible to the others.

The word that decides which move is available to you is **stateless**. A stateless server keeps nothing between requests that it cannot reconstruct or fetch — every request carries or looks up everything it needs. Statelessness is what makes servers interchangeable, and interchangeable is what makes "add another one" work at all. It is not a property you get for free; it is a discipline, and 1.3 is about paying for it.

Not all work scales this way, and it is worth being honest about which. Serving web requests does: they are independent. A database does not, not simply — it

state, which is why data gets its own five-chapter part later and why it will still be hard when it gets there.

So: start a second application server. Both are running, both are healthy, both are ready.

And nothing sends either of them any traffic, because your DNS record and your reverse proxy both point at exactly one address. You have doubled your capacity and served precisely as many requests as before.

Components introduced: none — a second instance of `app-server`. Reusing a component rather than inventing one is itself the lesson.

Get it wrong. Scaling vertically forever because it requires no thought, until you meet the ceiling during your busiest week and discover that the horizontal move requires code changes you now have no time to make.

Remember

- Vertical: no code changes, hard ceiling, single failure domain.

- Horizontal: linear capacity, survives failures, demands statelessness.
- Two servers with nothing in front of them are one server with a spare.

1.2 — Load Balancing

In one line: the box that makes "more machines" mean something.

A **load balancer** accepts every client connection and hands each request to one of several backends. That is the entire idea, and it resolves 1.1's cliffhanger in one component: clients still know one address, the fleet behind it is now free to change size.

How it chooses. The default is **round-robin** — take turns. It is right when requests cost roughly the same, and it is right more often than people expect. **Least-connections** sends work to whichever backend is currently handling the fewest requests, and is the better choice when request cost varies wildly, because a server stuck on three expensive uploads will otherwise keep receiving its share of cheap ones. **Hashing** on some request attribute sends the same key to the same backend every time, which you need only when a backend genuinely benefits from seeing the same traffic — a warm local cache, say. Reach for it reluctantly; it reintroduces the coupling horizontal scaling was meant to dissolve.

How it knows. This is the part that actually earns the component. A load balancer continuously **health checks** its backends — a cheap periodic request to something like `/health` — and removes any that stop answering. Without this it will cheerfully route a third of your users into a dead process. With it, a machine can die at 3am and the only evidence is a dip on a graph.

Health checks are also the first traffic in this book that is not the user's. Nobody asked for them; they carry no product data; they flow on their own schedule. That deserves its own name — a **control** edge, as distinct from the `request-flow` edges of Unit 0. You will meet three more edge kinds before the end, and each one marks a genuinely different kind of relationship.

Two things to be clear about. **L4** balancing routes by address and port, knowing nothing about the content; **L7** balancing reads the HTTP request and can route by path or header — more capability, slightly more cost. And a load balancer in front of one server is not a load balancer; it is a proxy with ambitions, buying you nothing but a hop.

Components introduced: `load-balancer`. **Edge kind:** `control`.

Get it wrong. A load balancer over a single instance (`single-instance-load-balancer`) — the shape of scaling without the substance. Or, more painfully: a fleet with no health checks, where a hung process keeps receiving traffic because it is still accepting connections.

Remember

- One address for clients, a changeable fleet behind it.
- Health checking is the feature; distribution is almost incidental.
- Round-robin unless request cost varies; least-connections when it does.

1.3 — Statelessness and Sessions

In one line: horizontal scaling quietly assumed your servers remember nothing, and login is the first thing that remembers.

Here is the bug, and almost everyone ships it once. A user logs in. Server A creates a session in its own memory and hands back a cookie. The next request goes — correctly, by round-robin — to server B, which has never heard of that session and returns them to the login page. The user logs in again, lands on A, and it works. The symptom is "I keep getting logged out at random", which is precisely as maddening to debug as it sounds.

The cause is that a load balancer made your servers interchangeable while one of them was still holding state. There are three ways out.

Sticky sessions tell the load balancer to pin each user to the server they first hit. It is one configuration change and it makes the symptom disappear, which is why it is so often chosen. It also quietly undoes what you bought. Traffic no longer distributes evenly — a server can be overloaded while its neighbours idle. Deploys become disruptive, because restarting a machine logs out everyone pinned to it. And you cannot scale down without evicting real users. Sticky sessions are a legitimate tool for genuinely expensive per-user setup; they are a poor tool for avoiding the work below.

Externalise the session. Move it out of process into a store every server can reach. Any server can then handle any request, and you are properly stateless again. For now that store is your database, which works and is slower than it should be — a session lookup on every single request is a lot of load for data that is small, hot, and disposable. There is a much better home for it two chapters away, and 2.2 moves it there.

Carry the state in the request. A signed token — a JWT is the common form — holds the user's identity and claims, and any server can verify the signature without looking anything up anywhere. Nothing to store, nothing to share. The cost is revocation: a token is valid until it expires because nothing is consulted, so "log out everywhere" and "ban this user immediately" stop being instant. Short lifetimes plus a refresh mechanism is the usual compromise, and it is a compromise.

Components introduced: none — `sql-database` doing a second job as a session store.

Get it wrong. In-memory sessions behind a load balancer. It passes every test on one machine and fails intermittently the moment there are two, which is the worst possible combination of properties.

Remember

- Any state in a process is invisible to its peers, and login is state.
- Sticky sessions trade away the benefits you just paid for.
- Externalise or carry: both work, and both cost something specific.

1.4 — Reverse Proxy vs. Load Balancer vs. API Gateway

In one line: three boxes that look identical on a diagram and answer three different questions.

You have now met two of these and the third is about to arrive. They are the most confused trio in system design, and the confusion is understandable — all three sit at the front, all three receive a request and forward it. What separates them is the question each one is answering.

A **reverse proxy** answers "where is this system?" One address, TLS terminated, internals hidden. It is about indirection.

A **load balancer** answers "which of my identical servers should handle this?" It presumes a pool of interchangeable backends and picks one, watching their health. It is about distribution.

An **API gateway** answers "is this request allowed, and which service does it belong to?" It presumes backends that are not interchangeable — several distinct services — and routes by what the request is asking for. It is about policy and dispatch.

The gateway earns its place when one backend became several. With three or ten services, three or ten separate implementations of authentication, rate limiting and request logging is not a design, it is a liability: they will drift, and one will drift into a hole. The gateway holds that shared policy once, at the edge — validate the token here, enforce the quota here, attach the request id here — so each service can assume the request reaching it has already been vetted.

Rate limiting in particular belongs here for a structural reason. It must run
before

work is done, at a point that sees all of a caller's traffic. Enforce it inside each service and you have neither property: the work is already partly done, and no single service knows the caller's total rate.

Be honest about the cost. A gateway is another hop, another thing to operate, another place a deploy can break everything at once. For a single service behind a load balancer it is overhead. The moment you have several services and cross-cutting policy, it stops being overhead and starts being the only sane place for that policy to live.

In practice one piece of software often plays several of these roles at once. The roles remain distinct even when the box does not — and knowing which role you are relying on is what lets you reason about what happens when it fails.

Components introduced: `api-gateway`.

Get it wrong. A gateway in front of one service, doing nothing but forwarding — cost without benefit. Or the reverse: several services, each with its own hand-rolled auth, one of which has a bug nobody has found yet.

Remember

- Proxy = indirection. Balancer = distribution. Gateway = policy and dispatch.
- Cross-cutting concerns belong at one edge, not duplicated per service.
- Rate limiting must sit where it sees everything, before the work starts.

PART III

Caching

The highest-leverage change in the book. Keep the answer near where it was asked, and decide how stale you can afford it to be.

2.1 — Cache-Aside

2.2 — Distributed Caching

2.3 — Caching at the Edge: CDN

Checkpoint R1 — A Site That Stays Up

2.1 — Cache-Aside

In one line: the highest-leverage change in this book — keep the answer near where it was asked.

Compute now scales. The database does not, and it is the same one database it was in Unit 0, now being hammered by an entire fleet. Before re-architecting your data — which is genuinely hard, takes five chapters, and is Part IV — try the cheap thing. Most systems read far more than they write, and most reads ask for the same small set of things over and over.

A **cache** is fast, in-memory key-value storage. **Cache-aside** — also called lazy loading — is the pattern for using one, and it lives in your application code, not in the cache:

1. Look in the cache. If the value is there (**hit**), return it. Done.
2. If it is not (**miss**), query the database.
3. Write what you got into the cache.
4. Return it.

Read that again with an eye on where the logic sits. The cache is passive — it stores what it is told and forgets on schedule. Your application decides what to cache, when to look, and what to do on a miss. This is why the pattern is robust: if the cache is unavailable, every request is a miss, and a miss is just the system you had in Unit 0. Slower, and alive.

TTL is the dial. Every entry gets a lifetime, after which it is discarded and the next reader pays for a fresh copy. TTL is not a performance setting; it is a **staleness budget**. Setting it to five minutes is a statement that you can tolerate serving data five minutes out of date. For a product catalogue that is fine. For an account balance it is a support ticket. Choose it from the data's meaning, never from a benchmark.

Hit rate is the outcome, and it is non-linear in a way worth internalising. Going from 0% to 50% halves database load. Going from 90% to 95% halves it again — the last few points are worth as much as the first fifty. This is why "what is actually hot?" is the question that matters more than any configuration.

Two things do not belong in a cache: data that is rarely read twice (you pay the write and never collect), and data that must be exactly correct at the instant it is read (you have chosen the wrong tool; go and make the read fast instead).

And then there is invalidation. When the underlying data changes, the cached copy is a lie until its TTL expires. You can wait it out, or delete the key on write, or overwrite it on write — and each of those has a race condition hiding in it, which is why the joke about invalidation being one of the two hard problems has survived so long. 2.2 makes it worse before it makes it better.

Components introduced: `cache`.

Get it wrong. Caching data whose correctness is non-negotiable, then discovering the staleness window during an incident. Or caching everything indiscriminately, so the hot data that would have paid for itself gets evicted by cold data that never gets read.

Remember

- Cache-aside logic lives in the application; the cache stays dumb, which keeps it optional.
- TTL is a staleness budget, chosen from what the data means.
- The last few points of hit rate carry as much load as the first fifty.

2.2 — Distributed Caching

In one line: a cache inside each server is several caches, and several caches disagree.

Put a cache in each application server's memory and you get spectacular numbers — no network hop at all. You also get a bug you can now predict from first principles, which is the point of this chapter arriving here rather than earlier.

Three servers, three private caches. A user updates their profile; the write lands on server A, which updates the database and refreshes its own copy. Servers B and C know nothing about it and keep serving the old profile until their entries expire. The user refreshes and sees the new name, refreshes again and sees the old one, refreshes again and sees the new one. They are being round-robin across three different versions of the truth.

Every ingredient of that failure came from a previous chapter — the load balancer spreading requests (1.2), state living inside one process (1.3), the cache holding a copy that can go stale (2.1). This is the first place the book cashes in on its own ordering, and it is the argument for the ordering.

The fix is to make the cache a component instead of a process detail. A **distributed cache** is a separate service every application server talks to over the network. One copy of each entry, one place to invalidate, and — importantly — cache contents that survive an application deploy, rather than every restart starting cold and stampeding the database.

You pay a network round trip, typically well under a millisecond in the same datacentre. In exchange all your servers see one truth. That is nearly always the right trade, and once you have made it, the store you built for it turns out to be exactly the fast session store 1.3 wanted and did not have. Sessions move here. Same component, second job.

Two failure modes deserve names. A **cache stampede** happens when a popular key expires and every server misses simultaneously, sending a thundering herd at the database for the same value — the mitigation is to let one request recompute while the others briefly serve stale or wait. And a distributed cache is now a shared dependency: when it is slow, everything is slow. Keep cache-aside's property that a miss still works, set aggressive timeouts, and never let a cache outage become a total outage.

Components introduced: `distributed-cache`.

Get it wrong. Per-instance caches behind a load balancer. It is fast, it passes review, and it produces intermittent wrong answers that correlate with nothing a log will show you.

Remember

- Per-process caches multiply your number of truths by your number of servers.
- One shared cache: one invalidation point, survives deploys, holds sessions too.
- Design for the cache being down; it must degrade to slow, not to broken.

2.3 — Caching at the Edge: CDN

In one line: the fastest request is the one that never reaches you.

Your cache sits next to your servers, so a hit still costs the user a round trip across the planet. Physics sets that floor: light through fibre from London to Sydney is roughly 80ms each way, and no amount of clever code beats it. The only way to make the trip shorter is to make it shorter.

A **CDN** is a network of caches in dozens or hundreds of locations worldwide. Users are steered to a nearby one — by DNS, exactly as promised in 0.2 — and if that edge holds the content, it answers directly. Your origin never learns the request happened.

Most CDNs work by **pull**: the edge fetches from your origin on its first miss for an object, caches it, and serves everyone else locally. You publish once to your own server; distribution is demand-driven and automatic. **Push** — uploading content to the edge in advance — exists for cases where even the first user must not pay the miss, such as a launch you know will be hit worldwide at a specific minute.

What belongs at the edge is anything identical for many users. Images, video, CSS, JavaScript, fonts, downloads — the bulk of the bytes in most products, and the part users perceive as "loading". Public API responses that change on a schedule qualify too. What does not belong is anything personalised: a page containing one user's name cached at a shared edge is the most embarrassing bug in this book, and it happens most commonly to someone who set a blanket cache header without thinking about who the response was for.

Control is by cache headers — how long an edge may keep an object, whether it may serve a stale copy while revalidating, whether it may store the response at all. And expiry has the same shape as DNS: you cannot easily un-send content already distributed. Explicit purges exist and propagate slowly. The standard answer is **versioned URLs**: give each build's assets a unique name, so a new deploy references new URLs and old ones simply age out. Never invalidate what you can rename.

Components introduced: `cdn`.

Get it wrong. Caching a personalised response at a shared edge, and serving one user's account page to everyone in their city. Runner-up: a long TTL on an asset at a stable URL, leaving you unable to ship a fix until it expires.

Remember

- The CDN wins on distance, which is the one cost you cannot optimise away.
- Shared content only; personalised responses must never be cached at a shared edge.
- Version URLs instead of purging them.

♦ Checkpoint R1 — A Site That Stays Up

Stop reading and build. Blank page, no starter, no new ideas — everything you need has been in the last nine chapters.

The brief: a mid-sized web product. Real traffic, more reads than writes, static assets and images, users who log in and expect to stay logged in, and an operations team that wants to deploy in the middle of the afternoon without anyone noticing.

Sketch the whole path from a user typing the name to a byte coming back, and be able to say out loud, for each box, what breaks if you delete it. A complete answer has a name that resolves, a perimeter with exactly one way in, a balanced fleet of stateless servers, session state living somewhere every server can see, a shared cache in front of the database, and static assets served from the edge.

Then answer three questions without looking anything up. Where does a user's session live, and why not in the server? When a cached profile is updated, what is the window during which somebody can still see the old one — and what sets its size? Which single component, if it fails, takes the entire site down with it?

If the third question has more than one answer, that is not a mistake in the exercise. Note them and carry them forward; Part VII is where you learn what to do about them.

PART IV

Scaling Data

Where scaling gets genuinely hard, because data is state and state cannot simply be duplicated and forgotten about.

3.1 — Read Replicas and Replication

3.2 — Choosing a Database: SQL vs. NoSQL

3.3 — Blobs: Object Storage

3.4 — Search: the Search Engine

3.5 — Partitioning and Sharding

3.1 — Read Replicas and Replication

In one line: copy the database so reads can spread out — and accept that copies are always slightly behind.

Caching bought time. It did not solve the shape of the problem: one database, every write, every uncacheable read, and no second machine if that one dies. Compute scaled in Unit 1 by adding identical servers. Data cannot do that trivially, because servers are interchangeable and data is not — but the first move in that direction is to make copies that are allowed to serve reads.

A **read replica** is a full copy of the primary database, kept up to date by a continuous stream of changes. Writes go only to the primary. Reads may go to any replica. Since most systems read far more than they write, this is a large win for one small change to your data access layer: two connections instead of one, and a decision at each call site about which to use.

That decision is the entire difficulty of the chapter. The stream of changes takes time, and during that time a replica is **behind** — usually milliseconds, occasionally seconds, and occasionally much longer when the primary is under heavy write load or the network is unhappy. This is **replication lag**, and it produces the single most recognisable bug in distributed data:

A user posts a comment. The write goes to the primary. The page reloads, the read is routed to a replica that has not received it yet, and their comment is gone. They post it again.

The property they expected is **read-your-own-writes**: whatever else is stale, my changes should be visible to me. The standard fix is not to eliminate lag — you cannot — but to route around it: after a user writes, send that user's reads to the primary for a short window. Other people's slightly stale view of their comment is nobody's crisis. Their own is.

The general rule follows from that. Route to a replica anything where being a second behind is invisible: browsing, search, analytics, most listing pages. Route to the primary anything a user just changed, and anything a decision depends on — inventory at checkout, balance before a transfer, permission before an action.

Synchronous replication — the primary waits for a replica to confirm before acknowledging the write — removes the lag and adds latency to every write, plus a new failure mode where a sick replica can stall the primary. **Asynchronous** is the common default and is what everything above assumes.

Note also what replicas do

not

give you: relief from write load. Every write still lands on the primary, and now also has to be shipped to every copy. Replicas scale reads, redundancy and read-side availability. Writes are still Chapter 3.5's problem.

Components introduced: `read-replica` . **Edge kind:** `replication` .

Get it wrong. A replica with no replication stream attached (`orphan-read-replica`) — a database serving confidently frozen data. Or sending writes to a replica, which either fails outright or, worse, succeeds locally and is silently overwritten.

Remember

- Writes to the primary, reads to replicas, always asymmetric.
- Lag is not a bug to fix; it is a property to route around.
- Read-your-own-writes is the guarantee users actually notice losing.

3.2 — Choosing a Database: SQL vs. NoSQL

In one line: not a technology contest — a question about what your data is shaped like and how you will ask for it.

This decision gets argued religiously and decided badly. The useful framing is not "which is better" but "what does my data look like, how will it be queried, and which guarantees do I refuse to give up?"

Relational databases store rows in tables with a fixed schema and connect them by keys. Two properties matter. **Joins** let you assemble an answer from data you never anticipated needing together, which means queries you did not plan for are cheap to add later. And **transactions** let several changes commit or fail as one indivisible unit — the property that makes "debit one account, credit another" safe, and which nothing else in this book substitutes for.

NoSQL is not one thing; it is a family united mostly by dropping some of the above in exchange for something else. Document stores keep whole nested objects and let the shape vary per record. Key-value stores do nothing but get and put, extremely fast. Wide-column stores handle enormous write volumes with a rigid access pattern. Graph stores make relationship traversal the primary operation.

Ask three questions.

Is the data relational in fact? Not "could it be normalised", but: do you routinely need several entities together, joined in ways you cannot enumerate in advance? Orders and customers and line items are relational. A blob of event attributes is not.

Do you know your access patterns now? This is the sharpest divider. A relational schema serves queries you have not thought of yet. A NoSQL schema is designed around specific queries, and is superb at them and awkward at everything else. Certainty about access patterns is what makes NoSQL a good bet; uncertainty is what makes relational a good bet.

Do you need multi-record transactions? If money, inventory or bookings are involved, this usually answers the whole question by itself. Many NoSQL stores now offer transactions with caveats; read the caveats, they are the product.

Some honesty about scale, since it is the reason most often given: a well-indexed relational database on decent hardware handles far more than people assume — millions of rows and thousands of writes per second is unremarkable. "We chose NoSQL for scale" is, in the vast majority of systems, a decision made for a scale that never arrived, paying the flexibility cost the whole time. And most real products end up with both: relational for the transactional core, something else for a specific workload that genuinely differs.

Components introduced: `nosql-database`.

Get it wrong. Picking on reputation rather than access pattern, and discovering six months in that the report your business needs requires a join your store cannot do — so you write it in application code, badly.

Remember

- Relational when queries are unpredictable or transactions are required.
- NoSQL when access patterns are known, fixed, and worth designing around.
- Both is normal; "scale" is rarely the real reason and often the stated one.

3.3 — Blobs: Object Storage

In one line: files do not belong in a database.

Someone uploads a profile picture. It is data, you have a database, and there is a column type that accepts bytes. Every part of that reasoning is locally sensible and the result is a mess.

Databases are built for small structured records, queried by index, cached in memory, copied to every replica and written to every backup. A five-megabyte image is none of those things. Store a hundred thousand of them and your database is half a terabyte of bytes nobody ever queries. Backups take hours. Replication lags because every image crosses the wire to every replica. The memory that should be caching hot rows is full of JPEG.

Object storage is the right home. It stores arbitrary blobs under keys, is effectively unbounded, is priced per gigabyte at a fraction of database storage, and is durable by design. What it does not do is query: no joins, no filtering by attribute, no transactions. It gets and puts by key. That constraint is why it is cheap.

The pattern is **indirection**. Bytes go to object storage. The database keeps a small row — id, owner, filename, content type, size, upload time, and the storage key — and that row is what you query, join and index. Metadata in the database, bytes in the store.

Two moves make this properly good.

Serve through a CDN. Object storage sits in one region; your users do not. Put 2.3's CDN in front, and images are served from the edge, cheaply and fast, with your origin untouched. This is the standard shape for all user media, everywhere.

Upload directly. The naive flow sends the file to your application server, which forwards it to storage — your server spends its time and bandwidth as a pipe, and a few large uploads can occupy the workers that should be serving requests. Instead, have the server issue a **presigned URL**: a time-limited, single-purpose credential that lets the client upload straight to the store. Your server authorises the upload and never touches a byte of it.

Components introduced: `object-storage`.

Get it wrong. Blobs in database columns. It works at ten files and hurts at ten thousand, and by then the migration touches every read path you have.

Remember

- Metadata in the database, bytes in object storage, always.
- Fronted by a CDN for delivery; presigned URLs for ingest.
- No queries, no transactions — that is the trade that makes it cheap.

3.4 — Search: the Search Engine

In one line: the first time you deliberately keep the same data in two places.

Your product needs search — free text, ranked by relevance, tolerant of typos, filtered by a few facets. Your database offers `LIKE '%term%'`, which cannot use an index and therefore reads every row, every time. It is fine on ten thousand rows and catastrophic on ten million, and it cannot rank, stem, or handle a misspelling anyway.

A **search engine** is built on an inverted index: rather than storing documents and scanning them, it stores, for each term, the list of documents containing it. Finding every document mentioning a word is then a lookup rather than a scan, and because it also stores frequencies and positions it can rank — which is what actually makes search feel like search.

This introduces something genuinely new to the book: **derived data**. Until now every piece of information had exactly one home. Now the same facts live in the database (the source of truth) and in the search index (a shaped copy, existing to answer one kind of question fast). That is a legitimate and common pattern, and it comes with one permanent obligation: something has to keep the copy in step.

The obvious approach is to update both on write — save the record, then index it. This is a **dual write**, and it is broken in a way worth seeing clearly, because the same shape recurs for the rest of your career. The two writes are not one atomic operation. If the second fails, you have committed data that will never be searchable, and no error the user sees. If the search engine is merely slow, every write in your product is now as slow as your slowest index update. If it is down, you must choose between failing writes that had nothing to do with search, or silently accepting permanent inconsistency.

None of the ways out are available yet. You need something that accepts "index this" durably, returns immediately, and retries until it succeeds — which is a queue, and a queue is Part V. Note the discomfort; it is the exact reason the next part exists.

One consequence is worth stating plainly regardless of mechanism: a search index is **eventually consistent** with its source. Freshly created things are briefly unfindable. Design the interface so that is unremarkable — show a user the item they just created directly, rather than by searching for it.

Components introduced: `search-engine`.

Get it wrong. Dual writes with no durable retry: silent, permanent divergence between what exists and what can be found. Runner-up: making search the source of truth because it already has all the fields.

Remember

- `LIKE` scans; an inverted index looks up, and can rank.
- Derived data is fine; the obligation is keeping it in step.

- Dual writes have no atomicity — the fix is durable asynchronous work.

3.5 — Partitioning and Sharding

In one line: the end of the road — when one machine cannot hold the data, split it, and accept everything that becomes harder.

Replicas spread reads. Caching absorbed the hot ones. Every write still lands on one primary, and the whole dataset still lives on one machine. When that machine's disk, memory or write throughput is the ceiling, there is exactly one move left: stop keeping all the data in one place.

Sharding splits the data horizontally across independent databases — each holds a subset of the rows, and no shard knows about the others. Note there is no new component for this. A shard is a database; the architecture is the same boxes with a different rule about which one holds what. Sharding is configuration, and that is a genuine insight, not a technicality: nothing on your diagram changes, and almost everything about your queries does.

The shard key is the entire decision. It is the attribute that determines which shard a row lives on, it is chosen once, and changing it later means moving all your data. Two ways to map keys to shards:

Range partitioning assigns contiguous ranges — customers A-F here, G-M there. Range scans stay efficient, and the distribution is usually uneven. Partition by date and every write goes to the newest shard while older ones idle: you have sharded and still have one hot machine.

Hash partitioning hashes the key and takes the result modulo the shard count. Distribution is excellent and range queries are destroyed, since adjacent keys are deliberately scattered. Adding a shard also changes the modulus and therefore where nearly everything belongs, which is why consistent hashing exists — it limits the movement to roughly one shard's worth.

Pick a key with high cardinality, even access, and — most importantly — one that matches how you query. If your queries filter by user and you shard by user, every query touches one shard. Shard by region instead and every user lookup fans out to all of them.

What gets worse, permanently: **cross-shard queries** must hit every shard and merge results, so they are as slow as the slowest one. **Joins across shards** effectively stop existing. **Global uniqueness** requires a scheme, since no shard can see the others. **Transactions** across shards need distributed protocols that are slow and fragile. And **hot keys** — one celebrity, one enormous tenant — can overload a single shard no matter how good the hash was.

Which leads to the most valuable advice in the chapter: **do not shard until you must.** Every option in Part IV and this part is cheaper. Sharding is what you do when a bigger machine, read replicas, caching, archiving old rows and moving blobs out have all been exhausted.

Components introduced: none — a configuration of existing databases.

Get it wrong. Sharding by a low-cardinality or time-based key and recreating the hot machine you were escaping. Or sharding on a key that does not match your query filters, so every read fans out to every shard.

Remember

- The shard key decides everything and is expensive to change.
- Range for scans and uneven load; hash for even load and no scans.
- Cross-shard joins, transactions and uniqueness all get worse; shard last.

PART V

Asynchronous Work

Everything so far made the user wait. The second half of every real system is work that happens later.

4.1 — Queues and Workers

4.2 — When Work Fails: Dead Letter Queues

4.3 — Scheduled and Ephemeral Compute

4.4 — Streams and Events: Event Bus and Kafka

Checkpoint R2 — The Whole Backend

4.1 — Queues and Workers

In one line: stop making the user wait for work that does not have to happen now.

Everything so far has been synchronous: a request arrives, work happens, a response goes back, the user waits for all of it. Look at what a signup actually does — validate the input, write the user row, generate a welcome image, send a confirmation email, notify the analytics pipeline, warm a cache. Only the first two have anything to do with the answer the user is waiting for. The rest are being charged to their latency for no reason, and worse: if the email provider is having a bad afternoon, signup fails. A user could not create an account because a third party could not send a message they had not asked for yet.

A **message queue** holds work to be done. A **worker** is a process that takes items off the queue and does them. The request handler writes the user row, puts "send welcome email" on the queue, and responds. Total time: the two things that mattered. The email is sent a moment later by something else, and if the provider is down, the message waits.

The **async** edge kind marks this: the producer does not wait for the consumer, and does not learn the outcome. That is a genuinely different relationship from every arrow so far, which is why it gets its own name.

What you gain, beyond latency. **Buffering:** a traffic spike becomes a longer queue rather than a collapse — work arrives faster than you process it, the backlog grows, and it drains when the spike passes. **Isolation:** a failing downstream dependency stops being able to fail your request path. **Independent scaling:** if the queue is growing, run more workers, without touching your web tier.

What you must accept. The user gets no confirmation that the work succeeded, so the interface has to be honest — "we'll email you" rather than "email sent". Work now happens out of order. And there is a backlog you have to watch, because a queue that only grows is an outage that has not been noticed yet.

The rule for what belongs on the request path is simple: **does the response depend on it?** The user's own data must be written before you claim it was. Sending mail, resizing images, updating search indexes, generating reports, calling third parties, recomputing recommendations — none of that changes the answer, and all of it belongs behind a queue. Chapter 3.4's dual-write problem now has its solution: the write commits, an "index this" message goes on the queue, and a worker retries until the index is updated.

One thing you must design for from the first day. Queues deliver **at least once** — if a worker crashes after doing the work but before acknowledging the message, the message reappears and is processed again. This is not a rare edge case; it is the normal behaviour of every reliable queue, because the alternative is losing messages. So workers must be **idempotent**: doing the same job twice must have the same effect as doing it once. Check whether the email was already sent; write the thumbnail to a deterministic key; use the message id as a deduplication key. Get this wrong and duplicate charges are in your future.

Components introduced: `message-queue`, `worker`. **Edge kind:** `async`.

Get it wrong. Non-idempotent workers — the double-charge, the duplicate email — because at-least-once delivery was treated as a theoretical concern. Runner-up: no alarm on queue depth, so a stalled worker pool goes unnoticed for a day.

Remember

- If the response does not depend on it, it does not belong on the request path.
- Queues buffer spikes and isolate failures, and hide problems in a backlog you must watch.
- At-least-once is the default; idempotency is your responsibility, not the queue's.

4.2 — When Work Fails: Dead Letter Queues

In one line: some messages can never succeed, and without somewhere to put them they will try forever.

A worker takes a message and fails. That is normal and usually transient — a timeout, a brief outage — so the message goes back on the queue and is retried. Retrying is the correct default and it is why queues are reliable.

Now consider a message that will never succeed. Malformed input from an old release. A reference to a record that has since been deleted. A bug that throws on one specific value. This is a **poison message**, and the retry loop that saves you from transient failures is now a trap: take, fail, return, take, fail, return, forever. It consumes worker capacity permanently, it fills your logs with the same stack trace, and — because it sits at the head of the queue — it can block everything behind it.

A **dead letter queue** is the escape. Count how many times each message has been delivered; after a threshold — three or five, rarely more — stop retrying and move it to a separate queue. The main queue keeps flowing. The failure is preserved rather than discarded. And the DLQ's depth becomes one of the most valuable signals you have: it should be zero, so anything in it is a real defect with a real payload attached.

Retry **backoff** matters as much as the retry count. Retrying immediately and repeatedly against a struggling dependency is a **retry storm** — you detect that a service is overloaded and respond by sending it more traffic. Exponential backoff (wait one second, then two, then four, then eight) with a little random jitter gives the dependency room to recover and stops all your workers from retrying in lockstep.

A DLQ is not an archive; it is an inbox. Something in it means a human must look: read the payload, find the bug, fix it, and then either replay the messages or consciously discard them. A dead letter queue nobody watches is exactly as useful as no dead letter queue, with the additional disadvantage that everyone believes failures are being handled.

Components introduced: `dead-letter-queue`.

Get it wrong. A queue with no DLQ (`queue-without-dead-letter-queue`) — poison messages cycling forever, consuming capacity, drowning the logs. Or a DLQ with no alarm, quietly accumulating a month of lost work.

Remember

- Retry a bounded number of times, with exponential backoff and jitter, then dead-letter.
- DLQ depth should be zero; treat any non-zero value as a defect report.
- Every queue in production needs one, and needs someone watching it.

4.3 — Scheduled and Ephemeral Compute

In one line: four ways to run code, and reaching for the wrong one is a habit worth breaking.

By now you have three shapes of compute and a tendency to use the first for everything. Lay them out properly.

An **application server** is always running and responds to requests. It is right when a user is waiting for an answer. A **worker** is always running and consumes from a queue. It is right for work triggered by an event, done as soon as capacity allows. A **cron job** runs on a schedule regardless of what happened. And a **serverless function** runs on demand, briefly, with no process kept alive between invocations.

Cron is for work defined by time rather than by events: nightly cleanup of expired sessions, a weekly report, hourly reconciliation against a payment provider, daily archival of old rows. The distinguishing question is whether the work should happen even if nothing happened — a report is due on Monday whether or not anyone used the product.

Cron has two failure modes that catch everyone once. **Overlap:** a job scheduled every ten minutes that occasionally takes twelve now has two copies running, competing over the same rows. Under load it takes longer, so it overlaps more, so it takes longer. And **duplicate execution:** you have several servers for redundancy, the same cron entry is on each, and a job meant to run once runs five times. Both need a mechanism to guarantee that only one execution proceeds — a lock — and locks are Chapter 5.2. Note the gap; the book comes back to it deliberately.

Serverless functions trade control for elasticity. You supply a function; the platform runs it on demand, scales from zero to thousands of concurrent executions automatically, and charges only for execution time. For spiky, short, stateless, event-driven work — resize this image on upload, respond to this webhook, run this one endpoint that gets forty requests a day — the economics and the operational simplicity are both excellent.

The costs are specific. **Cold starts:** an invocation with no warm instance waits for one to initialise, which can be tens of milliseconds or several seconds depending on runtime and dependencies. **Duration limits** make long jobs impossible. **No local state** between calls, so anything remembered goes to a cache or store. **Connection pressure** is the subtle one: a thousand concurrent functions each opening a database connection will exhaust a database that was comfortable with fifty from a fixed fleet. And the economics invert at steady high volume — per-invocation pricing is a bargain for bursty traffic and expensive for a constant load a small always-on fleet would serve for far less.

Serverless functions are usually fronted by an API gateway (1.4), which handles routing, auth and rate limiting for something that has no persistent process to hold them.

Components introduced: `cron-job`, `serverless-function`.

Get it wrong. A cron job with no overlap protection, running twice against the same rows. Or serverless chosen for a steady high-volume workload, where it costs several times what a small fleet would.

Remember

- Waiting user → server. Event → worker. Clock → cron. Spiky and short → function.
- Cron must be protected against overlap and multi-host duplication.
- Serverless: brilliant for bursty, expensive for constant, and watch the connections.

4.4 — Streams and Events: Event Bus and Kafka

In one line: a queue gives each message to one consumer, and sometimes that is exactly wrong.

The queue in 4.1 has a defining property: each message is delivered to exactly one worker. That is what you want for a task — one email, sent once. It becomes the obstacle the moment several

different

things must all react to the same occurrence.

An order is placed. Billing must charge the card, inventory must decrement stock, email must send a confirmation, analytics must record the sale, the warehouse must be told to pick it. With a queue you either have one consumer doing all five — recreating the monolith you were escaping — or the producer writes to five queues, meaning it must know all five consumers, and adding a sixth means changing the producer. The coupling you were trying to remove has simply moved.

An **event bus** inverts it. The producer publishes one event — "OrderPlaced" — describing something that happened, and knows nothing about who cares. Consumers subscribe. Each subscriber gets its own copy. Adding a sixth consumer requires no change to the producer at all. This is **publish/subscribe**, and it is the difference between commanding ("send this email") and announcing ("an order was placed"). Announcing is what decouples systems.

Kafka — a distributed log — takes it further by changing what is stored. A queue holds messages until they are consumed and then deletes them. A log holds an ordered, immutable sequence of records and keeps them for a configured retention period whether or not anyone read them. Each consumer tracks its own **offset**: its position in the log.

That single change buys three things nothing else gives you. **Replay**: a new consumer can start from the beginning and process all history — which is how you rebuild a broken search index, or launch a feature that needs the last year of events. **Independent consumers**: a slow one falls behind without affecting the others, because each holds its own offset. **Ordering**: within a partition, records are strictly ordered, so events for the same entity are processed in sequence.

Kafka's partitions are worth connecting to 3.5, because it is the same idea on a different substrate. A topic is split into partitions, records are assigned to one by a key, order is guaranteed only within a partition, and each partition is consumed by one member of a consumer group. So partition by the entity whose order matters — key by user id and that user's events stay in sequence — and the same hot-key warning applies: one enormous producer overloads one partition.

Choosing between the three: **queue** for tasks, when one consumer should do the work and the message is disposable afterwards. **Event bus** for notifications, when several independent things react and history does not matter. **Kafka** when you need replay, strict ordering, or very high sustained throughput — and are willing to operate it, because it is meaningfully more machinery than the other two.

Components introduced: `event-bus` , `kafka` .

Get it wrong. A queue where you needed fan-out, so one consumer accretes every downstream concern. Or Kafka where a queue would do, adding an operational burden nobody on the team can carry at 3am.

Remember

- Queue: one consumer, message consumed. Bus: many consumers, each a copy. Log: retained, replayable, ordered per partition.
- Publish facts, not instructions — announcing decouples, commanding does not.
- Kafka partitions are shards, with the same key-choice and hot-key problems.

✦ Checkpoint R2 — The Whole Backend

Blank page again, and this time nearly everything you have learned should appear.

The brief: an online shop. Customers browse a catalogue, search it, add to a basket, place orders, and get an email receipt. Product images and reviews are on every page. Finance needs a report every morning covering the previous day. Traffic triples for six hours during a sale, and the sale date is known in advance.

Build the whole thing, then walk one order from click to receipt and name every component it touches. Then walk one product page from the browser's DNS lookup to the last byte rendered.

The judgement calls are the point. Which reads may go to a replica, and which must go to the primary? What happens to an order placed at the exact moment the email provider goes down — and what does the customer see? Where does the morning report run, and what stops it from running twice? When the sale starts and traffic triples, what saturates first, and what did you build that turns that saturation into a growing backlog instead of an outage?

If you can answer those five without hedging, Part VI will be the hardest part of the book and you will be ready for it.

PART VI

Distributed Coordination

The hardest material, last on purpose: not plumbing but agreement, and what agreement costs.

5.1 — Leaders and Followers

5.2 — Agreement: Coordinators and Locks

5.3 — Trade-offs at the Core: Consistency and Availability

5.1 — Leaders and Followers

In one line: when several machines hold the same data, exactly one of them must be allowed to decide.

Chapter 3.1 introduced replication as mechanism: copies exist, they lag, route reads accordingly. This chapter asks the harder question that mechanism concealed — who is allowed to accept a write, and how does everyone agree on the answer? That is not plumbing. That is coordination, and it is why this part comes last.

The standard arrangement: one **leader** accepts all writes and streams them to several **followers**, which serve reads and stand ready to take over. One writer means no conflicts, and a single, well-defined order of events — which is exactly the property that makes the whole thing comprehensible.

The interesting part is what happens when the leader dies. **Failover** has three steps: detect the failure, choose a new leader, and redirect writes to it. Each is harder than it sounds.

Detection

is a guess. You cannot distinguish a dead leader from a slow one, or from a healthy one behind a broken network link. You have a timeout, and a timeout is an opinion. Set it short and you will promote a new leader because of a two-second network hiccup; set it long and you are unavailable for writes while everyone waits politely.

Promotion

must choose a follower — ideally the most up to date, since anything the old leader acknowledged but had not yet replicated is simply lost when someone else is promoted.

Redirection

must reach everything that writes, including the old leader, if it comes back.

Which is the nightmare: **split brain**. The old leader was never dead, only unreachable. It is still running, still accepting writes from whichever clients can still see it, while the newly promoted leader accepts different writes from everyone else. Two divergent histories, both believing they are authoritative, and no automatic way to merge them — you will be choosing which customers' data to discard.

The defence is **quorum**: a node may only act as leader if it can see a strict majority of the cluster. Since two disjoint majorities cannot exist, at most one side of any partition can proceed, and the minority side correctly refuses to act. This is why cluster sizes are odd. Five nodes tolerate two failures and still form a majority of three. Six nodes tolerate the same two failures and add a nasty case: a clean 3-3 split has no majority at all, so nobody can lead. You paid for a sixth machine and bought a new way to be unavailable.

Notice what the minority does when it cannot form a quorum: it stops serving writes. That is a deliberate choice of consistency over availability, and 5.3 is where that choice gets its proper name.

Components introduced: `leader`, `follower`.

Get it wrong. An even-numbered cluster, or leader election without quorum (`split-brain-risk`) — two leaders, two histories, and a data-loss decision made by a human under pressure.

Remember

- One writer removes conflict and gives you a single order of events.
- Failure detection is a timeout, and a timeout is a guess.
- Odd numbers and majority quorums; even numbers add failure modes, not safety.

5.2 — Agreement: Coordinators and Locks

In one line: sometimes exactly one process must do a thing, and "we'll be careful" is not a mechanism.

Two problems have been left open. In 4.3, a cron job on several hosts runs several times, and one that runs long overlaps itself. In 3.5, something has to know which shard owns which key range, and every client must agree. Both are the same question wearing different clothes:

how do independent processes agree on something?

A **lock service** answers the first. Before doing the exclusive work, a process asks for a named lock. Exactly one requester gets it; everyone else is refused and does nothing. The nightly job now runs once no matter how many hosts hold the cron entry, and cannot overlap itself, because the second copy cannot acquire what the first still holds.

The subtlety that makes distributed locking genuinely hard is this: **what if the holder dies?** A lock held forever by a crashed process blocks the work permanently. So locks have leases — they expire. Which creates the opposite hazard: a process that has stalled (a long garbage collection pause, a network partition, an unlucky scheduler) can believe it still holds a lock that has already expired and been granted to someone else. Now two processes are in the critical section, each certain it is alone. Mitigations exist — renew the lease, keep the work short, have the protected resource reject writes carrying an outdated lock generation — but the honest summary is that a distributed lock is a strong hint, not a law of physics, and any resource you truly cannot afford to have touched twice should also be idempotent.

A **coordinator** answers the second. It holds the small, critical, cluster-wide state everyone must agree on: which node is leader, which shards live where, which workers are alive, what the current configuration is. It is deliberately not a database — the data is tiny and changes rarely. What it provides is agreement, with strong consistency, so that no two clients ever see different answers. Underneath, it runs a consensus protocol over an odd-numbered cluster and survives a minority failing, exactly as 5.1 described.

Two warnings. A coordinator is a **dependency everything shares**, so it must be more reliable than anything relying on it, and clients should degrade gracefully when it is briefly unreachable — usually by continuing with their last known configuration rather than stopping. And coordination is expensive: every agreement is a round trip to a majority. Use it for decisions that are rare and critical, never on a per-request path.

Which is the real lesson of the chapter. Coordination is the most expensive thing in this book, and most designs that need it need it in one or two specific places. Find those places, pay for them properly, and keep everything else coordination-free.

Components introduced: `coordinator`, `lock-service`.

Get it wrong. Assuming a lock is absolute and building a non-idempotent operation on top of it. Or putting a coordinator in the request path and making every user request wait for a majority to agree about something.

Remember

- Locks make "exactly one" real; leases make them survivable and make them fallible.
- Coordinators hold small, critical, unanimously-agreed state — not application data.
- Coordination is expensive by nature; confine it to rare, critical decisions.

5.3 — Trade-offs at the Core: Consistency and Availability

In one line: the trade-off you have already made five times, finally given its name.

You have met this repeatedly without naming it: cache staleness in 2.1, cache invalidation in 2.2, replication lag in 3.1, eventual processing in 4.1, failover in 5.1. Every one was the same underlying choice, and it is time to say what it is.

CAP, stated honestly. When a network partition splits your system — and networks do partition, this is not optional — you must choose. Either refuse to serve requests on the side that cannot confirm it is current (**consistency**, at the cost of availability), or keep serving and accept that different sides may return different answers and diverge (**availability**, at the cost of consistency). The popular framing of "pick two of three" is misleading, because partition tolerance is not a design choice; it is a description of reality. The real statement is: when partitioned, choose C or A.

PACELC completes it, and is the more useful version day to day. If **P**artitioned, choose **A** or **C** — else, in normal operation, choose **L**atency or **C**onsistency. Even with a perfectly healthy network, a strongly consistent read must confirm with other nodes, and that confirmation takes time. This is why the trade-off shows up constantly, not just during incidents: strong consistency is slower on a good day too.

The essential move is that this is not a system-wide setting. It is a **per-operation** decision, and the same product will make it differently in different places:

A **payment or ledger** entry must be consistent. Two simultaneous withdrawals must not both succeed against one balance. Refuse the operation rather than risk it — an error message is recoverable, an incorrect balance is not.

A **social feed** should be available. If a post takes thirty seconds to appear for some users, nobody is harmed; if the feed is down, everybody notices. Serve slightly stale content, always.

A **shopping cart** is instructive because it is genuinely both. Adding items should always succeed — availability, with conflicts merged in favour of keeping items. Checkout, where stock is decremented and money moves, becomes consistent. One product, one flow, and the posture switches at a specific step.

A **presence indicator** — "online now" — should be available and is barely consistent by nature. The data is stale the instant it is produced, and nobody expects otherwise.

The design skill is to read a requirement and hear which one it is asking for. When a product person says "it must always work", they mean availability. When they say "it must never be wrong", they mean consistency. When they say both, your job is to find the boundary inside the flow where it changes — the cart/checkout line — rather than to pretend both are free.

Components introduced: none. This chapter has no shape on a diagram, which is precisely why it is the one worth re-reading.

Get it wrong. Choosing one posture for an entire system. Strong consistency everywhere buys unnecessary fragility and latency; eventual consistency everywhere eventually loses somebody's money.

Remember

- Partitions are not optional; the choice is what you do when one happens.
- PACELC: even healthy, consistency costs latency.
- Decide per operation, not per system — and find the line where a flow crosses over.

PART VII

Designing Whole Systems

The method itself — how you get from an ambiguous brief to a diagram you can defend line by line.

6.1 — From Requirements to Architecture

Checkpoint R3 — Open Brief

6.1 — From Requirements to Architecture

In one line: you have the components; this is the process for getting from a vague sentence to a diagram you can defend.

Every chapter so far taught a piece. Nobody has taught the
method

— what you actually do when someone says "design a ticket-booking site" and looks at you expectantly. There is a procedure, it has four stages, and the most common failure is skipping straight to stage three.

Stage 1 — Requirements

Do not draw anything yet. The brief is underspecified, always, and the first job is to make it specific.

Functional requirements are what the system does, written as user-visible capabilities: browse events, see seat availability, hold a seat while paying, complete a purchase, receive a ticket. Keep this list short and ruthlessly ordered. "Recommendations" and "gift cards" are real features that are not the problem you were asked about, and a design that tries to cover everything covers nothing well. Say out loud what you are excluding.

Non-functional requirements are the properties the system must have, and they are what actually determine the architecture. Scale, roughly. Read/write ratio. Latency expectations. Availability. Consistency, per operation, in 5.3's language. Durability — what may never be lost.

For ticket booking, the non-functional list already dictates most of the answer: reads vastly outnumber writes (everyone browses, few buy), browsing may be slightly stale, and a seat must never be sold twice. That last requirement is the system. Everything interesting will orbit it.

Stage 2 — Estimation

Order of magnitude only. The purpose is not accuracy; it is finding out which problems are real. Precision here is theatre — nobody's guess at daily active users is right, and being wrong by 3× rarely changes a decision, while being wrong by 1000× changes everything.

Ask three questions. **How many requests per second, at peak?** Take a daily figure, divide by 86,400 for the average, and multiply by five or ten for peak, since traffic is never flat. **What is the read/write ratio?** A 100:1 ratio says caching and replicas do most of your work. A 1:1 ratio says your write path is the design. **How much data, and how fast does it grow?** Gigabytes fit on one machine comfortably; terabytes need a plan; petabytes need a different book.

For our site: tens of millions of browse requests a day, tens of thousands of bookings. Overwhelmingly read-heavy, with a small write path that must be perfectly correct — and a brutal spike shape, because a popular event goes on sale at a published minute and the entire day's traffic arrives in ninety seconds.

Stage 3 — High-level design

Now

draw. Start with the shape from Checkpoint R1 — it is the correct skeleton for almost everything — and place components in the order the request travels: entry, edge, compute, then data, then anything asynchronous.

Justify each box in one sentence as you place it, and if you cannot, take it out. The CDN is there because event images and pages are identical for all users. The replicas are there because of the 100:1 ratio. The queue is there because confirmation emails must not sit on the booking path. A component you cannot justify in a sentence is a component you have copied from a diagram, and it will be the one you cannot defend when questioned.

Stage 4 — Deep dive

A high-level design is where a weak answer stops. The real work is picking the one subsystem that the requirements stress hardest and going deep on it.

Here it is unambiguous: **two people must not buy the same seat**. Everything else is standard. So walk it properly. The seat has states — available, held, sold. A hold is exclusive and expires, so an abandoned checkout does not remove the seat from sale forever. Acquiring a hold is exactly 5.2's problem, and it needs a real mechanism: a lock, or a conditional update that succeeds only if the seat is still available. Two simultaneous attempts, one wins, one is told clearly and immediately. Consistency wins here (5.3) even though availability wins everywhere else on the site.

Then attack it. What if the user's browser closes mid-payment? The hold expires and the seat returns. What if the payment provider times out but the charge actually went through? The booking operation must be idempotent and keyed to the attempt, or you will double-charge — 4.1's lesson at its most expensive. What happens in the ninety-second onsale spike? The read path is cached and the write path is small, deliberately, because it is the one that cannot be relaxed.

That is the method. Requirements before estimation, estimation before drawing, and drawing before depth — and the deep dive is where you demonstrate that you understand your own design.

Components introduced: none. The new thing is the workflow itself, and it is the most portable thing in this book.

Get it wrong. Drawing first. It produces a diagram that is generic, unjustifiable box-by-box, and silent about the one hard part — which is the only part anyone was interested in.

Remember

- Requirements, then estimation, then high-level, then deep dive. In that order.
- Non-functional requirements determine the architecture; functional ones determine the boxes.
- Estimate to an order of magnitude, and only to find out which problem is real.
- Every component gets a one-sentence justification or it does not get placed.

✦ Checkpoint R3 — Open Brief

The last checkpoint, and the first with no right answer.

The brief is deliberately underspecified:

design a system where users can follow each other and see what the people they follow have posted.

That is all you get. No scale, no latency target, no consistency requirement.

The change here is the posture. Until now there has been a shape you were building toward. From this point on there are many valid designs, and the standard is different: **anything that is not wrong passes**. Nothing prescribes push versus pull for the feed. Nothing requires a particular store. What fails is what embodies an anti-pattern this book has named — a client wired to a database, an even-numbered cluster, a queue with no dead letter queue, a personalised response cached at a shared edge, blobs in a database column.

So do the work of Chapter 6.1 first. Write the requirements you were not given, and state them as assumptions — "I am designing for ten million users, read-heavy, feeds may be a few seconds stale." Those assumptions are now part of your answer, and a reviewer should be able to disagree with an assumption without your design collapsing.

Then build, justify every box in a sentence, and name the one subsystem you would go deep on. If someone asks why you chose what you chose and your answer is a trade-off with a named cost on both sides, you have finished Building Blocks.

Part VIII is five real systems. There are no more prescribed shapes.

PART VIII

Five Real Systems

bit.ly, Instagram, a log collector, WhatsApp and Netflix — everything recombined, with no prescribed shapes left.

RWE-1 — bit.ly, a URL Shortener

RWE-2 — Instagram, Photo Sharing

RWE-3 — A Distributed Log Collector

RWE-4 — WhatsApp, Messaging

RWE-5 — Netflix, Video Streaming

RWE-1 — bit.ly, a URL Shortener

The system: turn a long URL into a short code, and redirect anyone who visits the short one. Two operations. It is the smallest complete system worth designing, which is why it is first.

What makes it interesting is asymmetry. Shortening happens rarely; redirecting happens constantly — ratios of hundreds or thousands to one are normal. So essentially all the engineering belongs on a read path that does one thing: take a short code, find a long URL, respond. That path should touch as little as possible.

Generating the code. Two approaches, and the trade-off is worth having properly. **Hashing** the long URL and truncating gives you a code with no coordination — any server can compute it alone — but truncation means collisions, so you must check whether the code is taken and handle it when it is. A **counter** — take the next integer, encode it in base62 — cannot collide and produces short codes densely, but it needs a single source of increasing numbers, which is coordination (5.2) on a write path. The standard trick avoids the round trip: give each server a block of ten thousand ids at a time, so it coordinates once per ten thousand writes and hands out ids locally in between. Note also that sequential codes are guessable, which matters if anyone will shorten something they consider private.

The read path is where the asymmetry pays. A code maps to a URL, the mapping never changes, and it is tiny — this is nearly the perfect cache entry. Client → load balancer → application → distributed cache, and on a miss, the store. Hit rates are extreme because link popularity is brutally skewed: a handful of links carry most traffic.

Choosing the store is a rare case where the NoSQL argument is genuinely strong (3.2). The access pattern is exactly one query — get by key — forever. There are no joins, no unpredictable reports, and eventually an enormous number of rows. A key-value or document store is designed for precisely this. Relational works too, indexed on the code, and is entirely reasonable at sub-billion scale.

Redirect semantics are a design decision people miss. A **301** (permanent) lets browsers and intermediaries cache the redirect, so subsequent visits never reach you at all — cheapest possible, and you lose visibility into those clicks and can never change the destination. A **302** (temporary) means every visit comes to you: you see every click and can change or expire a link. If analytics are part of the product, 302 is the answer, and you have deliberately chosen to pay for traffic you could have avoided.

Stretch — a link goes viral. One code takes a hundred times its usual traffic. Because the entry is tiny and cached, the distributed cache absorbs it; the database barely notices. This is the answer, and it is worth pausing on: the design already handled it because the read path was built around one hot cached lookup.

Stretch — click analytics without slowing redirects. Recording every click synchronously puts a database write on the fastest path in the system. Publish an event instead (4.1) and let a worker aggregate it. The user gets the redirect at full speed; the counting happens behind them.

Reinforces: load balancing and stateless tiers (1.2), cache-aside and distributed caching (2.1, 2.2), replicas and store choice (3.1, 3.2), async work (4.1).

RWE-2 — Instagram, Photo Sharing

The system: users upload photos, follow each other, and see a feed of recent posts from people they follow.

What makes it interesting is that it is the first design with genuinely separate subsystems. Upload, delivery and feed have different shapes, different bottlenecks and different failure modes. Each is familiar on its own; composing them is the lesson.

The upload pipeline is 3.3 in motion. The client uploads directly to object storage using a presigned URL — your servers never handle the bytes. A record is written with the metadata and storage key. A message goes on a queue. Workers then produce the derivatives every real product needs: thumbnail, feed-sized, full-sized, each in a modern format. Those go back to object storage, and the record is updated to say they are ready.

Everything about that is asynchronous on purpose. Processing takes seconds; the user should wait for none of it. The interface shows the upload as complete and fills in the processed image when it exists. Workers must be idempotent (4.1) — write derivatives to deterministic keys, so a retried job overwrites rather than duplicates.

Delivery is 2.3 doing what it was foreshadowed for. Images come from a CDN, always. This is the overwhelming majority of your bytes, it is identical for every viewer, and it is the difference between a fast product and an expensive one.

The feed is the real problem, and it is a genuine fork with no correct answer.

Fan-out on write (push). When someone posts, immediately write that post's id into a precomputed feed list for every one of their followers. Reading a feed is then a single lookup of an already-assembled list — extremely fast, which is right, because reads vastly outnumber writes. The cost is the **celebrity problem**: an account with forty million followers turns one post into forty million writes. That is a spike your system must absorb, and it delays delivery for the followers at the end of the queue.

Fan-out on read (pull). Store posts once. When someone opens their feed, fetch recent posts from each account they follow and merge. Writes are trivial and celebrities cost nothing. Reads are now expensive and get worse the more people you follow — and reads are the common operation, so you have optimised the rare case.

The real answer is hybrid, and it is worth knowing as a specific technique: push for ordinary accounts, pull for the small number of very-high-follower accounts, and merge the two at read time. Most of the feed is precomputed; the handful of celebrity posts are fetched live. Both pathologies are avoided by refusing to use one strategy for everyone.

Reinforces: object storage and CDN (3.3, 2.3), queues and workers (4.1, 4.2), NoSQL for feed data (3.2), distributed cache (2.2).

RWE-3 — A Distributed Log Collector

The system: thousands of machines emit log lines; collect them centrally, make recent logs searchable in seconds, keep older ones cheaply, and let engineers query both.

What makes it interesting is that it is the only system here with no end user and no browser — infrastructure, serving engineers. It also inverts the ratio every other project has: this is overwhelmingly **write-heavy**. Millions of lines a second arrive; queries are occasional. Every instinct built on read-heavy systems is wrong here.

Ingestion. An agent on each machine tails log files and ships batches. Batching is not an optimisation, it is the design — a network round trip per log line would cost more than producing the line did. Agents buffer locally and retry, so a brief collector outage does not lose data.

The spine is Kafka (4.4), and this is the case where it is unambiguously correct rather than fashionable. It absorbs enormous sustained write throughput. It retains records for a configured period whether or not anything has consumed them, which decouples producers from every consumer completely. Several independent consumers read the same stream at their own pace. And replay exists, which is the property that saves you: when the indexing logic has a bug, you fix it and reprocess from an earlier offset rather than losing a day of logs.

Partition by source — host or service — so one machine's lines stay ordered and load spreads. Watch for the hot partition (3.5, 4.4): one service logging ten times more than the rest overloads whichever partition it hashes to.

Hot path and cold path is the central structural idea, and it is a direct application of "different data, different store". Recent logs — the last few days, the ones anyone actually searches during an incident — go to a **search engine** (3.4), indexed and queryable in seconds. Older logs go to **object storage** (3.3), compressed, partitioned by time and source, costing almost nothing per terabyte and queryable slowly by batch job. Two consumers of the same Kafka stream, two very different stores, chosen by how the data will be accessed at each age.

Retention is a first-class design decision, not an operational afterthought. Storing everything forever is unaffordable at this volume, so someone must decide how long indexed logs stay indexed and how long archives are kept — often with different answers per source, and occasionally with legal constraints. A cron job (4.3) enforces it, and a lock (5.2) stops it running twice.

Backpressure is the failure mode to reason about. When ingest exceeds capacity, something must give. The queue absorbs a burst by growing — that is its job. A sustained overload is different: consumers fall behind, the lag grows monotonically, and eventually records are deleted by retention before anyone read them. The choices are to scale

consumers, to shed load deliberately (sample the least valuable sources), or to accept the loss. The one unacceptable option is not knowing, which is why consumer lag is the metric this system lives by.

Reinforces: Kafka (4.4), workers and DLQs (4.1, 4.2), object storage (3.3), search (3.4), cron (4.3), partitioning (3.5).

RWE-4 — WhatsApp, Messaging

The system: users exchange messages in real time, one-to-one and in groups, with delivery receipts and presence, and messages must survive the recipient being offline.

What makes it interesting is that it inverts the assumption this book has treated as sacred since Chapter 1.3. Real-time messaging needs **long-lived connections** — the server must push to a client that did not ask — and a connection is state. The server holding your connection is not interchangeable with any other server, because your connection is there.

This is exactly why the project comes last. The inversion only teaches you something if you thoroughly internalised the original.

Connections and routing. Clients hold a persistent connection to a connection server. The system needs a registry — user id to current connection server — which is small, hot, frequently updated and read on every message delivery: a distributed cache or coordinator (2.2, 5.2). To deliver a message, look up where the recipient is connected and route it to that server, which writes it down the socket. Nothing else about your architecture changes; the stateless tier still exists for everything that is not a live connection.

Offline delivery is the durability half, and the more important one. If the recipient is not connected, the message must be stored until they are. Per-user queueing: persist the message, deliver on reconnect, delete once acknowledged. Note that this makes delivery an asynchronous problem with retries, which means 4.1's rules apply in full.

Delivery guarantees are the product feature here — sent, delivered, read, the three ticks users understand precisely. Getting them right is an at-least-once problem (4.1). The client retries an unacknowledged send, so the same message can arrive twice; every message therefore carries a client-generated id, and the server deduplicates on it. Without that, a flaky connection produces duplicated messages, which users notice immediately and forgive slowly.

Ordering matters within a conversation and nowhere else. Sequence numbers per conversation let a client detect a gap and request what it missed. Global ordering across all conversations is neither needed nor affordable.

Group fan-out is the same push/pull decision as RWE-2 with different constants. Groups are small — hundreds, not millions — so fan-out on write is straightforward: one message becomes N deliveries, each with its own state per recipient.

Presence is where 5.3 gets applied honestly. "Online" is derived from heartbeats on the control edges introduced back in 1.2 — the client pings, the server records a timestamp, absence of a ping for some seconds means offline. It is stale by construction, it is high-volume, and it is completely unimportant: nobody is harmed by a two-second-old presence indicator. So it is availability-biased, kept in a cache, never persisted, and dropped under load before anything else.

Stretch — a region's connection tier fails. Every client in it reconnects at once. The **reconnect storm** is the real danger: a synchronised retry from millions of clients will flatten whatever they reconnect to. Clients need exponential backoff with jitter (4.2) so reconnection spreads over minutes rather than arriving as one wall.

Reinforces: queues and idempotency (4.1), delivery semantics (4.4), leader/follower and failover (5.1), consistency postures (5.3), gateway (1.4), control edges (1.2).

RWE-5 — Netflix, Video Streaming

The system: users browse a catalogue, pick something, and watch it — at high quality, without buffering, from anywhere.

What makes it the capstone is breadth. Three nearly independent systems must coexist in one design: an ingest and transcode pipeline, a browse and search experience, and a playback delivery network. Each reprises earlier work at higher stakes, and the exam is whether you can hold all three without letting one contaminate another.

Ingest and transcode is RWE-2's media pipeline, larger. A source file arrives — enormous — and must be transcoded into a ladder of resolutions and bitrates, then segmented into small chunks of a few seconds each, so a player can switch quality mid-stream as bandwidth changes. This is a long-running, parallelisable, retry-heavy batch job: queue plus a worker fleet (4.1), idempotent by segment key, with a dead letter queue that a human actually reads (4.2), writing to object storage (3.3). It is not on any user's critical path, and it should be nowhere near your request tier.

Browse and search is the familiar read-heavy web system, and it is where everything from Parts I to IV applies unchanged: edge, load balancer, stateless tier, distributed cache, replicas, search engine. Recommendations are computed offline by batch jobs and served as precomputed rows, because computing them per request would be a per-user latency cost for data that changes daily.

Playback delivery is the part that is genuinely different, and the economics drive everything. Video is essentially all of your bytes — several orders of magnitude more than everything else in this book combined. **Practically all of it must be served from the edge.** A CDN here is not an optimisation to add later; it is the product's cost structure. If a meaningful share of playback traffic reaches your origin, you have not built a slow system, you have built an unaffordable one.

Origin shielding follows from the same arithmetic. Hundreds of edge locations each missing on the same new release means hundreds of identical fetches from your origin. A shield tier sits between edges and origin, absorbing those misses so the origin serves each segment roughly once. And because you know in advance what will be popular, you **push** rather than pull (2.3): pre-position a major release at the edges before it launches, so no user ever pays for a miss.

Regional thinking arrives properly here. Users are worldwide; catalogue and playback should be served near them; viewing history and account state must be coherent for a user who travels. This is 5.3 at global scale: a resume position that is a few seconds stale across regions is completely acceptable, while a billing state that is inconsistent is not. Notice that this requires no new components — it is placement and consistency posture applied to what you already have.

Reinforces: everything. CDN (2.3) finally carrying the load it was introduced for, media pipeline (RWE-2), search and browse (3.4, 2.1, 3.1), consistency postures (5.3), async work (4.1, 4.2), and the whole of Part I on every request that is not a video byte.

The closing argument. Take this design and trace each part back to the chapter that taught it. Every box has one. That is what the ordering of this book was for: not a tour of components, but a sequence in which each new idea arrived at the moment the previous one had created a problem it could solve.

APPENDICES

Reference

The 27 components, every anti-pattern this book names, and the nine trade-offs that recur in almost every design.

[Appendix A — The 27 Components](#)

[Appendix B — Anti-Patterns](#)

[Appendix C — Reading the Trade-offs](#)

Appendix A — The 27 Components

Every component in the curriculum, where it is introduced, and what it is for.

#	Component	Ch.	What it does
1	client	0.1	The user's device. Untrusted by definition.
2	app-server	0.1	Where business rules, authn and authz are enforced.
3	sql-database	0.1	Durable structured state, joins, transactions.
4	browser	0.2	The client that resolves names and honours cache headers.
5	dns	0.2	Name to address, cached everywhere, steers coarsely.
6	firewall	0.3	Decides what is reachable from outside. Config, not topology.
7	reverse-proxy	0.3	One front door: TLS termination, one address, hidden internals.
8	load-balancer	1.2	Distributes across identical backends; health-checks them.
9	api-gateway	1.4	Routes to distinct services; holds cross-cutting policy.
10	cache	2.1	Fast key-value store fronting a slower one.
11	distributed-cache	2.2	One shared cache all servers see. Also the session store.
12	cdn	2.3	Edge caches near users. Shared content only.
13	read-replica	3.1	Copy of the primary serving reads, always slightly behind.
14	nosql-database	3.2	Store designed around known access patterns.
15	object-storage	3.3	Cheap unbounded blobs by key. No queries.
16	search-engine	3.4	Inverted index: full-text search and ranking. Derived data.
17	message-queue	4.1	Work to be done; one consumer per message.
18	worker	4.1	Process that consumes a queue. Must be idempotent.
19	dead-letter-queue	4.2	Where poison messages go after bounded retries.
20	cron-job	4.3	Work triggered by the clock, not by an event.
21	serverless-function	4.3	On-demand ephemeral compute. Bursty work, cold starts.
22	event-bus	4.4	Publish/subscribe: every subscriber gets a copy.
23	kafka	4.4	Retained ordered log. Replay, offsets, partitions.
24	leader	5.1	The one node permitted to accept writes.
25	follower	5.1	Replica that serves reads and stands for election.
26	coordinator	5.2	Small critical cluster state everyone must agree on.
27	lock-service	5.2	Makes "exactly one process does this" real.

Edge kinds. `request-flow` (0.1) — a request travelling to be answered. `control` (1.2) — traffic nobody asked for: health checks, heartbeats. `replication` (3.1) — changes streaming from a primary to a copy. `async` (4.1) — the producer does not wait and does not learn the outcome.

Sharding has no component (3.5), and neither does CAP (5.3). Both are configuration and judgement respectively. Not everything that matters is a box.

Appendix B — Anti-Patterns

The failure shapes named in this book, and the chapter that explains each.

Anti-pattern	Ch.	Why it breaks
Client wired directly to a database	0.1	Credentials published; no rule enforceable; schema becomes a public contract.
DNS used as a live failover mechanism	0.2	TTLs cannot be revoked; caches keep the old answer for as long as they were told to.
Permissive firewall	0.3	Perfect topology, fatal config. One compromise becomes total access.
Vertical scaling as the only plan	1.1	Hard ceiling, non-linear cost, and one machine is one failure.
Load balancer over a single instance	1.2	The shape of scaling without the substance — a hop for nothing.
No health checks	1.2	Traffic keeps routing into a process that is accepting connections and answering nothing.
In-memory sessions behind a load balancer	1.3	Passes on one machine; fails intermittently on two. Random logouts.
Sticky sessions used to avoid the work	1.3	Uneven load, disruptive deploys, cannot scale down without evicting users.
Gateway in front of one service	1.4	Cost, hop and failure domain with no cross-cutting policy to justify it.
Caching data that must be exactly current	2.1	The staleness budget you set is the wrongness you serve.
Per-instance caches behind a load balancer	2.2	N servers, N truths; users are round-robinbed between versions of reality.
Cache outage becomes total outage	2.2	A miss must degrade to slow, never to broken.
Personalised response at a shared edge	2.3	One user's page served to a whole city.
Long TTL on an asset at a stable URL	2.3	You cannot ship the fix until it expires. Version URLs instead.
Orphan read replica	3.1	A database serving confidently frozen data.
Writing to a replica	3.1	Fails outright, or succeeds locally and is silently overwritten.
Store chosen by reputation	3.2	The query your business needs later is the one your store cannot do.
Blobs in database columns	3.3	Backup bloat, replication lag, memory that should hold hot rows holding JPEG.

Anti-pattern	Ch.	Why it breaks
Dual writes with no durable retry	3.4	Silent permanent divergence between what exists and what is findable.
Sharding on a time-based or low-cardinality key	3.5	You sharded and still have exactly one hot machine.
Shard key that does not match query filters	3.5	Every read fans out to every shard.
Non-idempotent worker	4.1	At-least-once delivery is the normal case. Duplicate charges follow.
Unwatched queue depth	4.1	A stalled consumer pool is an outage nobody has noticed yet.
Queue with no dead letter queue	4.2	Poison messages retry forever, consuming capacity and blocking the queue.
Immediate retries with no backoff	4.2	You detect overload and respond by sending more traffic.
Cron with no overlap or duplicate protection	4.3	The same job runs twice against the same rows.
Serverless for steady high volume	4.3	Per-invocation pricing is a bargain when bursty and a bill when constant.
Queue where fan-out was needed	4.4	One consumer accretes every downstream concern.
Even-numbered cluster	5.1	A clean split has no majority: nobody can lead.
Leader election without quorum	5.1	Split brain — two histories, and a human choosing whose data to discard.
Trusting a distributed lock absolutely	5.2	Leases expire while a stalled holder still believes it holds one.
Coordination on the request path	5.2	Every user request now waits for a majority to agree.
One consistency posture for the whole system	5.3	Strong everywhere is fragile and slow; eventual everywhere eventually loses money.
Drawing before requirements	6.1	A generic diagram, silent about the one hard part anyone cared about.

Appendix C — Reading the Trade-offs

Nine decisions that recur in almost every design, with the question that resolves each.

Vertical or horizontal? Is a hard ceiling or a single failure domain acceptable? (1.1)

Round-robin or least-connections? Do requests cost roughly the same? (1.2)

Sticky sessions or externalised state? Is per-user setup genuinely expensive, or are you avoiding the work? (1.3)

What TTL? How stale may this data be before someone is harmed? Never a benchmark question. (2.1)

Replica or primary for this read? Did this user just change this data, or does a decision depend on it? (3.1)

SQL or NoSQL? Do you know your access patterns, and do you need multi-record transactions? (3.2)

On the request path or behind a queue? Does the response depend on it? (4.1)

Queue, bus, or log? One consumer or many; and do you need replay or ordering? (4.4)

Consistent or available? Per operation, not per system — and where inside the flow does the answer change? (5.3)